

컴퓨팅 알고리즘 수학 커리큘럼 요약

5단원 완전 정복 가이드 — 알고리즘 효율성 분석부터 정보 이론까지

목차

- I. 알고리즘 효율성 분석
- II. 고급 선형대수
- III. 계산 기하학 기초
- IV. 수치 해석 및 수치 연산
- V. 정보 이론
- 전체 큰 그림

I. 알고리즘 효율성 분석

알고리즘을 "자원을 얼마나 쓰는지 수학적으로 증명 가능한 절차"로 바라보는 단원.
"이 로직이 데이터 늘어나면 버틸까?"를 정량적으로 답할 수 있게 됩니다.

1. 증가 차수와 점근적 표기법

핵심 요약

입력 크기 n 이 커질 때 실행 시간이 **얼마나 빠르게 증가하는가**만 보겠다는 관점. 상수배와 저차항은 무시하고 본질적인 성장률만 비교합니다.

세 가지 표기법

표기	의미	설명
$O(g(n))$	상한 (upper bound)	최악이라도 이거보단 빠르다
$\Omega(g(n))$	하한 (lower bound)	아무리 잘해도 이거보단 오래 걸린다
$\Theta(g(n))$	정확한 차수	O 와 Ω 를 동시에 만족, 딱 이 정도다

엄밀한 정의: $O(g(n))$ — 어떤 상수 c, n_0 가 존재해서 모든 $n \geq n_0$ 에 대해 $f(n) \leq c \cdot g(n)$

작은 o , 작은 ω

- $o(g(n))$: f 가 g 보다 엄격히 느리게 증가 $\rightarrow \lim f/g = 0$ (" O 는 $\leq$, o 는 $<$ ")
- $\omega(g(n))$: f 가 g 보다 엄격히 빠르게 증가

자주 헛갈리는 포인트

- $\log$ 의 밑은 의미 없음 (상수배 차이로 점근적으로 같음)
- $n \log n < n^{1.01}$ (다항식이 결국 $\log$ 를 이김)
- 2^n 과 $n!$ 은 천지차이 ($n!$ 이 더 빠르게 폭발)

2. 마스터 정리를 이용한 재귀 분석

핵심 요약

$T(n) = aT(n/b) + f(n)$ 꼴의 분할 정복 재귀식을 공식 하나로 푸는 도구. 머지소트, 이진탐색, 스트라센 행렬곱이 다 이 패턴.

변수 의미

변수	의미
a	한 번 재귀에서 만드는 부분 문제 개수
n/b	각 부분 문제의 크기
$f(n)$	분할하고 합치는 데 드는 비용

세 가지 케이스 (비교 기준: $n^{(\log_b a)}$)

케이스	조건	결론
Case 1	$f(n) = O(n^{(\log_b a - \epsilon)})$	$T(n) = \Theta(n^{(\log_b a)})$ — 앞 비용이 지배
Case 2	$f(n) = \Theta(n^{(\log_b a)})$	$T(n) = \Theta(n^{(\log_b a)} \cdot \log n)$ — 모든 레벨 균등
Case 3	$f(n) = \Omega(n^{(\log_b a + \epsilon)})$	$T(n) = \Theta(f(n))$ — 루트 비용이 지배

주요 예시

알고리즘	점화식	결과
머지소트	$T(n) = 2T(n/2) + n$	$\Theta(n \log n)$
이진탐색	$T(n) = T(n/2) + 1$	$\Theta(\log n)$
스트라센	$T(n) = 7T(n/2) + n^2$	$\Theta(n^{2.807})$

3. 분할 상환 분석 (Amortized Analysis)

핵심 요약

"한 번은 비싸지만 평균적으로는 싸다"를 증명하는 기법. 동적 배열 push가 대표 예시 — 가끔 $O(n)$ 이 튀어도 전체 평균은 $O(1)$.

세 가지 분석 기법

기법	방법	특징
총합법 (Aggregate)	총비용 $T(n)$ 을 n 으로 나눔	가장 직관적
회계법 (Accounting)	연산마다 예치금 부과, 비싼 연산이 꺼내 씀	유연한 설계
위치 에너지법 (Potential)	자료구조 상태를 Φ 로 매핑, 분할상환 비용 = 실제 + $\Delta\Phi$	가장 강력하나 Φ 설계 어려움

실전 등장처

동적 배열 push, Union-Find path compression, 스프레이 트리, 피보나치 힙

4. 평균 케이스 및 확률적 분석

핵심 요약

입력 분포를 가정하고 기댓값을 계산하는 평균 분석.
알고리즘 안에서 무작위성을 쓰면 확률적 알고리즘.

평균 케이스 vs 확률적 알고리즘

구분	무작위성 출처	예시
평균 케이스 분석	입력이 무작위 분포	퀵소트 (무작위 입력 가정)
확률적 알고리즘	알고리즘 내부의 난수	무작위 피벗 퀵소트, 밀러-라빈 소수 판정

두 종류의 확률적 알고리즘

종류	답	시간	예시
라스베가스	항상 정확	확률 변수	무작위 피벗 퀵소트

종류	답	시간	예시
몬테카를로	확률적으로 틀릴 수 있음	결정적	밀러-라빈 소수 판정

핵심 도구: 기댓값의 선형성

$E[X+Y] = E[X] + E[Y]$ — X, Y 가 독립이 아니어도 성립. 지표 확률변수를 잘게 쪼개 합으로 만든 뒤 각각의 기댓값을 더하는 패턴이 평균 분석의 핵심.

II. 고급 선형대수

벡터와 행렬을 "숫자 배열"이 아닌 **공간을 변환하는 도구**로 보는 단원. 머신러닝, 그래픽스, 추천 시스템, 검색엔진이 다 이 위에서 동작합니다.

1. 벡터 공간과 부분 공간

핵심 요약

벡터 공간 = "덧셈과 스칼라곱이 닫혀 있는 집합". 실수 벡터뿐 아니라 **함수, 다항식, 행렬도 벡터 공간**이 될 수 있어요.

핵심 개념 4종 세트

개념	정의
선형 결합	$c_1v_1 + c_2v_2 + \dots$ 형태
생성 (Span)	벡터들의 모든 선형 결합 집합
선형 독립	$c_1v_1 + \dots + c_nv_n = 0$ 의 유일한 해가 모든 $c=0$
기저 (Basis)	선형 독립이면서 공간 전체를 생성하는 벡터 집합

행렬의 네 가지 기본 공간

공간	정의	차원
열공간 (Column Space)	A 의 열벡터들의 span	$\text{rank}(A)$
영공간 (Null Space)	$Ax=0$ 의 해집합	$n - \text{rank}(A)$
행공간 (Row Space)	A 의 행벡터들의 span	$\text{rank}(A)$
좌영공간 (Left Null Space)	$A^Ty=0$ 의 해집합	$m - \text{rank}(A)$

2. 고윳값과 고유벡터의 컴퓨팅 응용

핵심 요약

$A\mathbf{v} = \lambda\mathbf{v}$ — 행렬 A 를 작용시켜도 방향이 안 바뀌고 크기만 λ 배 되는 특별한 벡터. 행렬의 "본질적 축"을 찾는 작업.

구하는 방법

- 특성 방정식: $\det(A - \lambda I) = 0 \rightarrow$ 고윳값 λ 구하기
- 각 λ 에 대해 $(A - \lambda I)\mathbf{v} = 0$ 풀기 $\rightarrow$ 고유벡터 $\mathbf{v}$

대각화

n 개의 선형 독립 고유벡터가 있으면: $A = PDP^{-1} \rightarrow A^n = PD^nP^{-1}$ (거듭제곱이 매우 빨라짐)

스펙트럼 정리 (대칭 행렬 $A^T = A$)

- 모든 고윳값이 실수
- 서로 다른 고윳값의 고유벡터는 직교
- 직교 대각화 가능: $A = QDQ^T$

컴퓨팅 응용

분야	내용
PageRank	웹 링크 행렬의 최대 고유벡터 = 페이지 중요도
PCA	공분산 행렬의 고유벡터 = 데이터의 주축
스펙트럴 클러스터링	그래프 라플라시안의 작은 고윳값으로 군집 분리
진동 분석	강성 행렬의 고윳값 = 고유진동수 ²

3. 특이값 분해(SVD)와 데이터 차원 축소

핵심 요약

$A = U\Sigma V^T$ — 임의의 직사각 행렬도 "회전 $\rightarrow$ 늘리기 $\rightarrow$ 회전"으로 분해. 추천 시스템, 이미지 압축, 노이즈 제거의 핵심.

구성 요소

행렬	크기	의미
U	$m \times m$ 직교행렬	좌특이벡터들
Σ	$m \times n$ 대각행렬	특이값 $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$
V^T	$n \times n$ 직교행렬의 전치	우특이벡터들

고유분해 vs SVD

구분	고유분해	SVD
대상	정사각 행렬만	임의의 직사각 행렬
항상 존재?	아니오	예 — 모든 행렬에 존재

Truncated SVD — 차원 축소

상위 k 개 특이값만 남기면: $A \approx A_k = U_k \Sigma_k V_k^T$

■ **에카르트-영 정리:** A_k 는 랭크 k 행렬 중 A 와 가장 가까운 행렬 (최적 압축)

실제 응용

분야	내용
이미지 압축	상위 50개 특이값만 → 100배 압축, 시각적 거의 동일
추천 시스템	사용자×영화 평점 행렬 분해 → 잠재 요인 추출
LSA	문서×단어 행렬 분해 → 의미 공간 구성
PCA	데이터 중심화 후 SVD와 동일
유사역행렬	$A^+ = V \Sigma^+ U^T \rightarrow$ 최소제곱법의 정해

4. 행렬 연산의 최적화 알고리즘

핵심 요약

교과서 행렬곱은 $O(n^3)$. 분할 정복과 메모리 계층 인식으로 더 빠르게.
작은 차이가 GPU 시대에 엄청난 차이가 됩니다.

주요 알고리즘 비교

알고리즘	복잡도	특징
교과서 방식	$O(n^3)$	삼중 루프
스트라센	$O(n^{2.807})$	7번 곱셈으로 분할 정복 (n=1024부터 유리)
Coppersmith-Winograd	$O(n^{2.376})$	이론적
현재 최고	$O(n^{2.371552})$	갈락틱 알고리즘 (실용성 없음)

분해 선택 가이드

상황	추천 분해
대칭 양정치 행렬	Cholesky (LU의 2배 빠름)
일반 정사각 시스템	LU
최소제곱 / 직사각	QR 또는 SVD
차원 축소 / 추천	Truncated SVD
그래프 분석	고유분해 (스펙트럴)

III. 계산 기하학 기초

게임 엔진의 충돌 처리, 지도 앱의 경로 탐색, GIS, 로봇 경로 계획, 그래픽스까지.
"점, 선, 다각형"이 데이터고, "교차, 포함, 거리"가 연산입니다.

1. 기하 알고리즘의 기초 (외적과 방향성)

핵심 요약

계산 기하의 거의 모든 게 **외적(cross product)** 위에서 굴러가요. 점이 직선의 어느 쪽인지, 세 점이 시계 방향인지 — 외적 부호 하나로 다 판정.

2D 외적

$$\mathbf{u} \times \mathbf{v} = \mathbf{u}_1 \cdot \mathbf{v}_2 - \mathbf{u}_2 \cdot \mathbf{v}_1 \text{ (스칼라값)}$$

부호의 의미

결과	의미
양수	반시계 방향 (CCW)
음수	시계 방향 (CW)
0	평행 또는 일직선

CCW 판정 (세 점 A,B,C)

$$cross = (B.x - A.x)(C.y - A.y) - (B.y - A.y)(C.x - A.x)$$

양수 → 반시계 (CCW) / 음수 → 시계 (CW) / 0 → 일직선

선분 교차 판정 (외적 4번으로 해결)

$$d1 = ccw(C, D, A), \quad d2 = ccw(C, D, B)$$
$$d3 = ccw(A, B, C), \quad d4 = ccw(A, B, D)$$

$(d1 \cdot d2 < 0) \text{ AND } (d3 \cdot d4 < 0) \rightarrow \text{교차}$

2. Convex Hull 알고리즘

핵심 요약

볼록 껍질(Convex Hull) = 점들을 모두 포함하는 가장 작은 볼록 다각형.
"고무줄로 점들을 둘러싸면 만들어지는 모양"

주요 알고리즘 비교

알고리즘	시간 복잡도	특징
Jarvis March	$O(nh)$	h 는 껍질 점 개수, 껍질 작으면 유리
Graham Scan	$O(n \log n)$	표준, 극각 정렬 후 CCW 스택
Andrew's Monotone Chain	$O(n \log n)$	구현 간결, x좌표 정렬
Quickhull	평균 $O(n \log n)$, 최악 $O(n^2)$	분할 정복, 실전 빠름
Chan's Algorithm	$O(n \log h)$	출력 민감, 이론적 최적

Graham Scan 핵심 로직

- y 좌표가 가장 낮은 점 P_0 선택 (무조건 꺾일 포함)
- P_0 기준 극각으로 나머지 점 정렬
- 스택 순회: CCW면 push, CW면 pop (오목해지는 점 제거)

하한: 비교 모델에서 정렬로 환원 가능 $\rightarrow O(n \log n)$ 이 하한

3. 점 분할 자료구조 (k-d 트리, 사분트리)

핵심 요약

공간을 재귀적으로 쪼개서 "근처 점 빨리 찾기" 가능하게 만드는 트리.
최근접 이웃 탐색(NN search)의 핵심.

k-d 트리

각 레벨마다 다른 축을 기준으로 공간을 반씩 분할하는 이진 트리.

- 2D: x, y 번갈아 / k차원: mod k 순환
- 구축: $O(n \log n)$ / 최근접 탐색: 평균 $O(\log n)$, 최악 $O(n)$
- 핵심: 현재 최단 거리 < 분할 평면까지 거리면 반대쪽 가지치기

사분트리 (Quadtree / Octree)

종류	차원	자식 수	용도
Quadtree	2D	4	게임 충돌, GIS, 이미지 압축
Octree	3D	8	3D 렌더링, 입자 시뮬레이션

차원의 저주

차원이 높아지면($d > 20$) k-d 트리의 가지치기가 무력화. 고차원 대안: **Ball Tree**, **LSH**, **HNSW**(벡터 검색 표준)

4. 최단 경로 탐색과 공간 가속화

핵심 요약

Dijkstra, A* 같은 알고리즘에 기하 정보를 더해 가속.

"서울에서 부산 길찾기"가 1초 안에 되는 이유가 여기 있어요.

기본 알고리즘

알고리즘	복잡도	특징
Dijkstra	$O((V+E) \log V)$	음수 가중치 없을 때 표준
A*	휴리스틱에 따라 다름	admissible + consistent 휴리스틱 → 최적 보장
Bellman-Ford	$O(VE)$	음수 가중치 허용
Floyd-Warshall	$O(V^3)$	모든 쌍 최단 경로

대규모 도로망 가속 기법

기법	가속 원리	적용 사례
Bidirectional Search	출발·도착 동시 탐색, 중간에서 만남	탐색 공간 절반
ALT	랜드마크 + 삼각 부등식으로 정확한 휴리스틱	A* 개선
Contraction Hierarchies	노드 수축 + 지름길 간선 전처리	OSRM, GraphHopper
Hub Labeling	허브 라벨 교집합으로 즉시 계산	나노초 수준

IV. 수치 해석 및 수치 연산

"수학적으로 맞는 게 컴퓨터에서 정확한 건 아니다."
유한한 정밀도 위에서 신뢰할 수 있는 결과를 뽑아내는 기법들.

1. 부동소수점 오차 분석과 안정성

핵심 요약

IEEE 754 double은 유효 15~17자리. 대부분의 실수를 정확히 표현 못 하고,
그 오차가 어떻게 누적되고 폭발하는지 이해해야 합니다.

오차의 종류

종류	원인	설명
반올림 오차	표현 한계	double 기계 엡실론 $\approx 2.22 \times 10^{-16}$
절단 오차	무한급수의 유한 근사	$\sin x$ 의 테일러 급수 근사

종류	원인	설명
누적 오차	반복 연산	작은 오차가 쌓임
상쇄 오차	비슷한 두 수의 뺄셈	유효 자릿수가 한꺼번에 날아감

핵심 구분

개념	대상	정의
조건수 κ	문제의 본질	입력 변화 대비 출력 변화율. 클수록 ill-conditioned
수치 안정성	알고리즘의 본질	오차가 계산 과정에서 증폭되지 않는 정도

$\kappa(A) = \sigma_{\max} / \sigma_{\min}$ (SVD의 특이값 비율!)

실무 팁

- 금융 계산: float 금지 → **Decimal 타입** 또는 정수(센트 단위)
- 비교: `a == b` 금지 → `abs(a-b) < epsilon`
- 큰 수 + 작은 수: 작은 것부터 더하기 (Kahan summation)

2. 비선형 방정식의 수치 해법

핵심 요약

$f(x) = 0$ 의 근을 수치적으로 찾는 방법들. 5차 이상 다항식은 일반 해 공식이 없어 반복 근사가 필수.

세 가지 대표 방법

방법	수렴 차수	수렴 보장	도함수 필요	특징
이분법	1 (선형)	O	X	가장 안전, 매 반복 오차 1/2
시컨트법	≈ 1.618 (황금비!)	X	X	도함수 없이 초선형
뉴턴-랩슨	2 (이차)	X	O	가장 빠름, 초기값 민감

뉴턴-랩슨 점화식

$$x_{n+1} = x_n - f(x_n) / f'(x_n)$$

실무 표준: Brent's method = 이분법(안전) + 시컨트법(빠름) 하이브리드 → SciPy의 `brentq`, MATLAB의 `fzero`

다변수 확장

벡터 방정식 $F(x) = 0$: $x_{n+1} = x_n - J(x_n)^{-1} \cdot F(x_n)$ (J 는 자코비안 행렬)

3. 수치적 적분 및 미분법

수치 적분 방법 비교

방법	오차	특징
사다리꼴 공식	$O(h^2)$	구간을 사다리꼴로 근사
심슨 공식	$O(h^4)$	포물선 근사, 같은 비용에 훨씬 정확
가우스 구적법	$2n-1$ 차 다항식 정확	최적 표본점과 가중치 사용
적응적 구적법	자동 조절	변동 심한 곳에 더 조밀 → SciPy <code>quad</code>
몬테카를로	$O(1/\sqrt{N})$	고차원 적분의 유일한 해법

수치 미분의 딜레마

전진 차분: $f'(x) \approx (f(x+h) - f(x)) / h$

중심 차분: $f'(x) \approx (f(x+h) - f(x-h)) / (2h)$

→ 오차 $O(h)$

→ 오차 $O(h^2)$

h 가 크면 절단 오차 증가, h 가 작으면 상쇄 오차 폭발

최적 $h \approx \sqrt{\epsilon}$ (double에서 약 10^{-8})

자동 미분 (Automatic Differentiation)

연쇄 법칙을 코드 실행 중 자동 적용 — 수치 미분의 오차 문제 해결.

모드	유리한 경우	대표 응용
전진 모드	입력 수가 적을 때	—
역전파 모드	출력 수가 적을 때	신경망의 backpropagation

PyTorch, JAX, TensorFlow가 모두 자동 미분 기반

4. 행렬 방정식의 직접 해법과 반복 해법

핵심 요약

$Ax = b$ 푸는 방법들.

 "역행렬을 구하지 말고 풀어라" — 역행렬 계산은 거의 항상 나쁜 선택

직접 해법

방법	적용 대상	복잡도	특징
LU 분해	일반 정사각, dense	$O(n^3)$ 분해, $O(n^2)$ 재사용	b 바뀔 때 재사용 가능
Cholesky	대칭 양정치	$O(n^3)/2$	LU보다 2배 빠름, 더 안정
QR 분해	직사각 / 최소제곱	$O(mn^2)$	최소제곱법의 표준
SVD	매우 나쁜 조건수	$O(mn^2)$	가장 안정적, 가장 느림

반복 해법 (대규모 희소 행렬용)

방법	적용 대상	특징
야코비	대각 우세	병렬화 쉬움, 수렴 느림
가우스-자이델	대각 우세	야코비보다 2배 빠름, 순차적
켈레기울기법 (CG)	대칭 양정치	현대 표준, $O(nnz)$ per 반복
GMRES, BiCGSTAB	비대칭	CG의 일반화

선택 가이드

상황	방법
$n < 10^4$, dense	LU/Cholesky 직접법
n 큼, 희소, 대칭 양정치	전처리된 CG
n 큼, 희소, 일반	전처리된 GMRES/BiCGSTAB
같은 A , 여러 b	LU 한 번 분해 후 재사용

V. 정보 이론

1948년 클로드 새넌이 단 한 편의 논문으로 만든 분야.

"정보란 무엇인가, 얼마나 많은가, 어디까지 압축 가능한가, 잡음 속에서 어떻게 복원하는가"

1. 엔트로피와 정보량의 측정

핵심 요약

정보량 = 놀라움의 정도. 확률이 낮은 사건일수록 일어났을 때 정보량이 큼.

정보량의 정의

$$I(x) = -\log_2 p(x) \text{ (단위: 비트)}$$

사건	확률	정보량
동전 앞면	1/2	1 비트
주사위 6	1/6	≈ 2.585 비트
확실한 사건	1	0 비트

엔트로피 (평균 정보량)

$$H(X) = -\sum p_i \cdot \log_2 p_i$$

분포	엔트로피
공정한 동전	1 비트
공정한 주사위	≈ 2.585 비트
실제 영문 텍스트	$\approx 1.0 \sim 1.5$ 비트/문자
완전 편향 동전	0 비트

최대 엔트로피 = 균등 분포일 때 ($H_{\max} = \log_2 n$)

엔트로피의 네 가지 해석 (모두 같은 값)

- 불확실성의 양
- 평균 정보량
- 최소 부호화 비트 수
- 스무고개 평균 질문 수

KL 발산

$$D_{KL}(P||Q) = \sum p(x) \cdot \log(p(x)/q(x))$$

머신러닝의 cross-entropy loss = KL 발산 최소화 분류 모델 학습 = 예측 분포 Q를 진짜 분포 P에 맞추기

2. 데이터 압축과 수학적 한계

핵심 요약

어떤 압축 알고리즘도 평균적으로 엔트로피보다 짧게는 못 만든다.

새넌의 소스 부호화 정리

$$H(X) \leq L < H(X) + 1$$
 (L = 평균 부호 길이)

허프만 부호화

가장 빈도 높은 심볼에 가장 짧은 부호 할당하는 욕심쟁이 알고리즘.

- 빈도 작은 두 노드를 반복적으로 묶어 이진 트리 구성
- 앞까지 경로가 부호 (0: 왼쪽, 1: 오른쪽)
- 평균 길이 $\approx H(X) \sim H(X)+1$

주요 압축 알고리즘

알고리즘	방식	적용 사례
허프만	확률 기반 트리	JPEG, MP3의 내부 단계
산술 부호화	전체 메시지를 실수 하나로	JPEG, H.264, HEVC
LZ77	반복 패턴 참조	gzip, zip, PNG
DEFLATE	LZ77 + 허프만 결합	zip, gzip, PNG
ANS	산술만큼 효율적이고 더 빠름	zstd, AV1
신경 압축	LLM으로 확률 모델링	압축 $\approx$ 예측 $\approx$ 지능

손실 vs 무손실

구분	한계 이론	예시
무손실	새넌 소스 부호화 정리 (엔트로피 한계)	zip, PNG, FLAC

구분	한계 이론	예시
손실	율-왜곡 이론 (Rate-Distortion)	JPEG, MP3, H.264

3. 오류 검출 및 교정 부호

핵심 요약

새넨 채널 부호화 정리: 채널 용량 C보다 작은 전송률이면 적절한 부호로 오류 확률을 0에 가깝게 만들 수 있다.

핵심 개념: 해밍 거리

두 비트열에서 다른 비트의 개수.
최소 거리 d_{min} 이면:

- $(d-1)$ 개 오류 검출 가능
- $\lfloor (d-1)/2 \rfloor$ 개 오류 정정 가능

주요 부호 정리

부호	능력	실제 적용
패리티 비트	1비트 오류 검출만	기본 데이터 전송
해밍(7,4)	1비트 정정	ECC RAM
CRC	강력한 오류 검출	이더넷, Wi-Fi, USB, ZIP, PNG
리드-솔로몬	연속 오류(burst) 정정	CD, DVD, QR 코드, RAID-6
컨볼루션 + 비터비	채널 오류 정정	휴대전화, 위성 통신
터보 부호	새넨 한계 0.5dB 이내	3G, 4G
LDPC	새넨 한계 근접	5G, Wi-Fi 6, NVMe SSD
폴라 부호	새넨 한계 수학적으로 도달	5G 제어 채널

QR 코드가 찢어져도 읽히는 이유

리드-솔로몬 부호 적용. L/M/Q/H 레벨이 RS 부호 비율을 의미.

🏠 전체 큰 그림

단원별 핵심 질문

단원	핵심 질문	핵심 도구
I. 알고리즘 분석	얼마나 빠른가?	점근적 표기, 마스터 정리, 분할 상환
II. 선형대수	데이터를 어떻게 변환하나?	고윳값, SVD, 행렬 분해
III. 계산 기하	공간을 어떻게 다루나?	외적, Convex Hull, k-d 트리
IV. 수치 해석	실수를 어떻게 정확히 계산하나?	안정성, 뉴턴법, LU 분해
V. 정보 이론	정보를 어떻게 측정·압축·복원하나?	엔트로피, 허프만, 해밍

분야별 단원 연결

분야	관련 단원
머신러닝	I(복잡도) + II(행렬 연산) + IV(수치 최적화) + V(cross-entropy)
그래픽스/게임	I(자료구조) + II(변환 행렬) + III(충돌·렌더링) + IV(시뮬레이션)
검색 엔진	I(인덱싱) + II(PageRank, LSA) + III(공간 인덱스) + V(압축)
암호학/통신	I(알고리즘) + IV(수치) + V(부호화)
금융 시스템	I(고빈도 알고리즘) + II(공분산) + IV(수치 적분, 몬테카를로)

이 커리큘럼 다 익히면, "왜 이 라이브러리가 이렇게 동작하는지",
"이 성능 문제의 근본 원인이 뭔지", "이 데이터를 어떻게 다뤄야 하는지"가 보이는 단계에 들어갑니
다.